determines the command name, you only ever end up with one class in the file. I typically get creative and call my class `Command`.

The key part of your `Command` class is the `.handle()` method, which Django calls when your management command gets run. It is best practice to use `*args` and `**options` in the signature of `.handle()`. For the first version of the `musicians` command, it isn't necessary, but stick with it for now, and you'll use these pieces in the next section when you add command-line arguments.

By inheriting from `BaseCommand`, you automatically get some features, like `help`. If you include a `help` attribute on your class, Django uses its value to populate the command-line help information.

Our `musicians` command lists all the `Musician` objects in the database, so the command imports the `MusicianModel`, performs a query on it, and outputs the result. It is recommended to use `self.stdout.write()` for output inside of a management command instead of `print()`. The `print()` function will work, but by using `self.stdout`, you can take advantage of the testing capabilities for commands, covered later. The following listing contains the code for the first version of the `musicians` command.

Listing 9.2 A management command that displays the Musician objects in the database

```
# RiffMates/bands/management/commands/musicians.py
from django.core.management.base import BaseCommand
from bands.models import Musician

class Command(BaseCommand):
    help = "Lists registered musicians"

    def handle(self, *args, **options):
        for musician in Musician.objects.all():
            self.stdout.write(
                f"{musician.last_name}, {musician.first_name}"
            )
```

Management commands are classes that inherit from BaseCommand.

Provide a help string for the --help command-line flag.

The .handle() method is the command's invocation point.

Query and loop through all Musician objects.

Output each Musician, using self.stdout.write() instead of print().

With the file in place, you're ready to go. First off, run `manage.py` without any arguments, and notice you now have a `bands` section with the `musicians` command inside of it:

```
(venv) RiffMates$ python manage.py

Type 'manage.py help <subcommand>' for help on a specific subcommand.

Available subcommands:

[auth]
    changepassword
    createsuperuser

[bands]
    musicians

...
```